

Module 19 — Automatisation Python (API & scripts)

Bootcamp Data Analyst — From Zero to Hero | Niveau Intermédiaire · Module 19

Ce que tu seras capable de faire à la fin de ce module

- Expliquer ce qu'est une API, les différents types qui existent (REST, SOAP, GraphQL, WebSocket), et pourquoi REST domine pour un DA
- Faire une requête GET et parser une réponse JSON, en Python et en R
- Repérer un piège réel : une API peut renvoyer une erreur **dans le corps de la réponse**, avec un statut HTTP 200
- Écrire une fonction réutilisable, paramétrable, avec gestion d'erreurs
- Journaliser (logger) le résultat d'un script à chaque exécution
- Comprendre comment planifier l'exécution automatique d'un script
- Envelopper un script automatisé dans une mini app **Streamlit**

 **Durée estimée** : 2h30  **Outils** : Python (`requests`, `streamlit`) · R (`http`, `jsonlite`)  **Prérequis** : Modules 14-15 (Niveau Intermédiaire)

1. Pourquoi automatiser

Jusqu'ici, chaque analyse que tu as faite était **ponctuelle** : tu charges un fichier, tu l'analyses, c'est fini. En poste, une bonne partie du travail d'un·e DA consiste à répéter **la même extraction et la même analyse**, régulièrement — le taux de change du jour, les ventes de la veille, l'état des stocks. Refaire ça à la main chaque matin n'est ni fiable ni un bon usage de ton temps.

Ce module te donne les trois briques pour automatiser ce type de tâche :

1. **Récupérer la donnée automatiquement** depuis une API, sans intervention humaine
2. **Écrire un script réutilisable**, qui gère proprement ses erreurs
3. **Planifier son exécution**, et éventuellement l'exposer via une petite app

2. Qu'est-ce qu'une API, et quels types existent

Une **API** (Application Programming Interface) est un point d'accès qu'un service met à disposition pour que d'autres programmes puissent lui parler — sans interface graphique, juste des requêtes et des réponses. Ce n'est pas un concept unique : plusieurs **styles d'API** coexistent, et tu peux croiser chacun d'eux en entreprise.

TYPE	PRINCIPE	OÙ TU LE CROISES
REST	Requêtes HTTP classiques (GET, POST...), réponses généralement en JSON	Le standard le plus répandu aujourd'hui — API météo, taux de change, réseaux sociaux, la grande majorité des APIs publiques
SOAP	Protocole plus ancien et plus rigide, messages en XML, souvent accompagné d'un contrat de service formel (WSDL)	Systèmes bancaires, gouvernementaux, grandes entreprises avec des systèmes historiques (legacy)
GraphQL	Le client précise exactement les champs qu'il veut dans une seule requête, plutôt que de récupérer une réponse figée	APIs plus récentes (ex. GitHub, Shopify) — évite de récupérer trop, ou pas assez, de données en un seul appel
WebSocket	Connexion permanente entre client et serveur — le serveur pousse des mises à jour en continu, pas de requête/réponse ponctuelle	Données en temps réel : cours de bourse en direct, chat, notifications

💡 **En tant que DA**, tu utiliseras REST dans l'immense majorité des cas — c'est le plus simple à consommer et le plus répandu pour de la donnée en lecture (statistiques, taux, indicateurs). Mais si un·e collègue mentionne "l'API GraphQL du CRM" ou "le webservice SOAP de la banque", tu sauras maintenant de quoi il s'agit, même sans l'avoir manipulé.

Zoom sur REST — celle qu'on utilise dans ce module

Une **API REST** répond à des requêtes HTTP (comme un navigateur web) et renvoie généralement les données au format **JSON**.

```
Ton script — GET https://exemple.com/api/taux —> Serveur de l'API
Ton script <— réponse JSON {"USD": 574.48, ...} — Serveur de l'API
```

C'est exactement ce que fait ton navigateur quand tu visites un site — sauf qu'ici, c'est ton script Python ou R qui joue le rôle du navigateur, et la réponse est pensée pour être lue par une machine plutôt que par un humain.

3. Faire une requête GET et lire le JSON

On utilise une API réelle, gratuite et sans clé : open.er-api.com, qui donne les taux de change actuels entre devises — un scénario réaliste pour un·e DA qui doit convertir des montants reçus en devises étrangères (USD, EUR...) en FCFA pour un reporting.

Python (`requests`) :

PYTHON

```
import requests

reponse = requests.get("https://open.er-api.com/v6/latest/USD")
print(reponse.status_code)    # 200 = succès

data = reponse.json()         # convertit le JSON en dictionnaire Python
print(data["rates"]["XOF"])   # le taux USD -> FCFA
print(data["time_last_update_utc"])
```

R (`httr` + `jsonlite`):

R

```
library(httr)
library(jsonlite)

reponse <- GET("https://open.er-api.com/v6/latest/USD")
status_code(reponse) # 200 = succès

data <- fromJSON(content(reponse, "text", encoding = "UTF-8"))
data$rates$XOF
data$time_last_update_utc
```

💡 Le taux USD → FCFA que tu obtiens **change chaque jour** (contrairement à EUR → FCFA, fixé par traité depuis la création du FCFA et donc toujours égal à 655,957). Relance ce code demain, tu obtiendras un chiffre différent pour USD — c'est le principe même de l'automatisation : le script reste identique, la donnée qu'il récupère évolue.

4. Un vrai piège — l'erreur qui ne dit pas son nom

Teste ton code avec un code de devise invalide :

PYTHON

```
reponse = requests.get("https://open.er-api.com/v6/latest/ZZZ")
print(reponse.status_code)    # 200 !
print(reponse.json())         # {'result': 'error', 'error-type':
                              'unsupported-code'}
```

Le statut HTTP est 200 (succès) alors que la requête a en réalité échoué — l'erreur est signalée **dans le corps de la réponse JSON** (`"result": "error"`), pas par le code de statut. Ce n'est pas un cas isolé : de nombreuses APIs fonctionnent ainsi.

⚠ **Réflexe indispensable** : ne te fie jamais uniquement au code de statut HTTP. Vérifie toujours le contenu de la réponse avant de l'utiliser :

PYTHON

```
def recuperer_taux(devise_source, devises_cibles):
    reponse = requests.get(f"https://open.er-
api.com/v6/latest/{devise_source}", timeout=10)
    reponse.raise_for_status()          # leve une exception si le statut
    HTTP est une erreur (4xx/5xx)
    data = reponse.json()
    if data.get("result") != "success": # verifie AUSSI le contenu, pas
    seulement le statut
        raise ValueError(f"L'API a renvoyé une erreur : {data.get('error-
type')}")
    return {devise: data["rates"][devise] for devise in devises_cibles if
    devise in data["rates"]}
```

R (même logique) :

R

```
recuperer_taux <- function(devise_source, devises_cibles) {
  reponse <- GET(paste0("https://open.er-api.com/v6/latest/", devise_source))
  stop_for_status(reponse) # equivalent de raise_for_status()
  data <- fromJSON(content(reponse, "text", encoding = "UTF-8"))
  if (data$result != "success") {
    stop(paste("L'API a renvoyé une erreur :", data[["error-type"]]))
  }
  data$rates[devises_cibles]
}
```

5. Journaliser le résultat à chaque exécution

Un script d'automatisation utile garde une trace de ce qu'il a récupéré à chaque exécution — un simple fichier CSV auquel on **ajoute une ligne** à chaque appel suffit largement.

Python :

```
PYTHON
import csv
import os
from datetime import datetime

def logger_taux(chemin_csv, devise_source="USD", devises_cibles=("XOF",
"EUR")):
    taux = recuperer_taux(devise_source, devises_cibles)
    ligne = {"horodatage": datetime.now().isoformat(), "devise_source":
devise_source, **taux}

    fichier_existe = os.path.exists(chemin_csv)
    with open(chemin_csv, "a", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=list(ligne.keys()))
        if not fichier_existe:
            writer.writeheader()
        writer.writerow(ligne)
    return ligne

logger_taux("historique_taux.csv")
```

Exécute ce script deux jours de suite : `historique_taux.csv` accumule une ligne à chaque fois, avec un horodatage — tu commences à construire un historique que tu pourras analyser plus tard (avec les outils du Module 15).

R (équivalent avec `write.table` en mode ajout) :

R

```
logger_taux <- function(chemin_csv, devise_source = "USD", devises_cibles =  
c("XOF", "EUR")) {  
  taux <- recuperer_taux(devise_source, devises_cibles)  
  ligne <- data.frame(horodatage = Sys.time(), devise_source = devise_source,  
t(taux))  
  
  fichier_existe <- file.exists(chemin_csv)  
  write.table(ligne, chemin_csv, sep = ",", append = fichier_existe,  
              col.names = !fichier_existe, row.names = FALSE)  
}
```

6. Planifier l'exécution automatique

Un script qu'il faut lancer soi-même chaque jour n'est qu'à moitié automatisé. Pour une vraie automatisation, il faut le **planifier**. Il existe quatre grandes façons de faire, selon ton environnement :

ENVIRONNEMENT	COMMENT PLANIFIER
Windows	Planificateur de tâches (Task Scheduler) — exécute ton script <code>.py</code> à une heure fixe, tous les jours
Mac/Linux	<code>cron</code> — une ligne dans la crontab (<code>0 8 * * * python3 /chemin/script.py</code>) = tous les jours à 8h)
Dans le script Python lui-même	La librairie <code>schedule</code> : <code>schedule.every().day.at("08:00").do(logger_taux, ...)</code> , puis une boucle qui vérifie régulièrement s'il est temps d'exécuter
Cloud	Un job planifié hébergé (GitHub Actions avec un déclencheur <code>cron</code> , par exemple) — ton script tourne même si ton ordinateur est éteint

Voyons chacune de ces quatre options en détail, avec le code ou la commande qui va avec.

6.1 Windows — Planificateur de tâches (Task Scheduler)

Sur Windows, l'outil natif pour exécuter un script automatiquement s'appelle le **Planificateur de tâches** (*Task Scheduler*). Il est déjà installé, gratuit, et suffit largement pour lancer un script Python chaque jour à heure fixe.

ÉTAPE 1 — OUVRIR LE PLANIFICATEUR DE TÂCHES

- Appuyez sur **Win + R**, tapez **taskschd.msc**, puis validez.
- Ou cherchez "Planificateur de tâches" dans le menu Démarrer.

ÉTAPE 2 — CRÉER UNE TÂCHE DE BASE

1. Dans le panneau de droite, cliquez sur **Créer une tâche de base...**
2. Donnez un nom explicite, par exemple **Extraction_rapport_quotidien**, et une description courte.
3. À l'étape **Déclencheur**, choisissez **Quotidienne (Daily)**.
4. Précisez l'heure de démarrage souhaitée, par exemple **06:30:00**, et laissez la récurrence à "tous les 1 jours".
5. À l'étape **Action**, choisissez **Démarrer un programme**.

ÉTAPE 3 — CONFIGURER L'ACTION (LA PARTIE QUI CASSE TOUT SI ON SE TROMPE)

C'est ici que se joue 90 % des problèmes. Trois champs à remplir séparément :

CHAMP	VALEUR À METTRE	EXEMPLE
Programme/script	Le chemin complet vers python.exe (idéalement celui de votre environnement virtuel)	C:\Users\utilisateur\projets\rapport\venv\Scripts\python.exe
Ajouter des arguments	Le chemin complet vers votre script .py	C:\Users\utilisateur\projets\rapport\main.py
Commencer dans (le "Start in")	Le dossier du projet, sans guillemets	C:\Users\utilisateur\projets\rapport

Ne mettez **jamais** tout dans un seul champ (**python main.py**) : le Planificateur attend un exécutable dans "Programme/script" et les arguments séparément.

LE CHAMP "COMMENCER DANS" — LE PIÈGE CLASSIQUE

Le champ "**Commencer dans**" (**Start in**) définit le *répertoire de travail* dans lequel le script sera lancé. Si vous le laissez vide, Windows exécute votre script depuis un dossier par défaut (souvent `C:\Windows\System32`).

Résultat concret : si votre script contient du code comme

```
PYTHON
df = pd.read_csv("data/ventes.csv")
```

il fonctionnera parfaitement quand vous le lancez à la main depuis VS Code ou un terminal (parce que vous êtes déjà dans le bon dossier), mais échouera silencieusement — ou plantera avec un `FileNotFoundError` — une fois lancé par le Planificateur, car le chemin relatif `data/ventes.csv` sera cherché dans `C:\Windows\System32\data\ventes.csv`, qui n'existe pas.

La solution : renseignez toujours le champ "Commencer dans" avec le dossier racine de votre projet. C'est le correctif le plus simple, et il évite 90 % des "ça marche sur ma machine mais pas dans la tâche planifiée".

ÉTAPE 4 — VÉRIFIER ET TERMINER

Cochez la case **Ouvrir la boîte de dialogue Propriétés...** avant de cliquer sur **Terminer**, pour accéder aux options avancées :

- Onglet **Général** : cochez **Exécuter que l'utilisateur soit connecté ou non** si vous voulez que la tâche tourne même si la session Windows est verrouillée.
- Onglet **Conditions** : décochez **Ne démarrer la tâche que si l'ordinateur est branché sur secteur** si vous testez sur un portable sur batterie.

ÉQUIVALENT EN LIGNE DE COMMANDE : `SCHTASKS /CREATE`

Pour aller plus vite, ou pour scripter la création de la tâche (utile si vous déployez la même automatisation sur plusieurs machines), on peut utiliser `schtasks` directement dans un terminal (PowerShell ou Invite de commandes, en administrateur) :

```
POWERSHELL
schtasks /create /tn "Extraction_rapport_quotidien" /tr
"C:\Users\utilisateur\projets\rapport\venv\Scripts\python.exe\"
"C:\Users\utilisateur\projets\rapport\main.py\" /sc daily /st 06:30 /rl
highest
```

Détail des options :

- `/tn` : nom de la tâche (*task name*)
- `/tr` : la commande à exécuter (*task run*) — si le chemin de l'exécutable ou du script contient des espaces, il doit être encadré par des guillemets doubles ; comme l'ensemble de l'argument `/tr` est lui-même déjà entre guillemets doubles, les guillemets internes doivent être échappés avec un antislash (`\\".\""`), et non remplacés par des guillemets simples (les guillemets simples n'ont aucune valeur de citation dans `cmd.exe` / PowerShell pour ce genre de chemin)
- `/sc daily` : fréquence quotidienne (*schedule*)
- `/st 06:30` : heure de démarrage (*start time*)
- `/rl highest` : exécute la tâche avec les droits élevés (équivalent administrateur)

Attention : `schtasks /create` en ligne de commande ne propose pas d'équivalent direct au champ "Commencer dans". Pour contourner ce manque, la solution la plus robuste est de gérer le répertoire de travail directement dans le script Python, en début de fichier :

```
PYTHON
import os

os.chdir(os.path.dirname(os.path.abspath(__file__)))
```

Cette ligne force le script à se placer dans son propre dossier avant de lire ou écrire le moindre fichier relatif, quel que soit l'endroit d'où il est lancé.

Pour vérifier que la tâche est bien créée :

```
POWERSHELL
schtasks /query /tn "Extraction_rapport_quotidien" /v /fo list
```

Et pour la supprimer :

```
POWERSHELL
schtasks /delete /tn "Extraction_rapport_quotidien" /f
```

LE GOTCHA À RETENIR : LE MAUVAIS `PYTHON.EXE`

Si vous travaillez avec un environnement virtuel (`venv` ou `conda`), l'erreur classique est de renseigner le `python.exe` global de Windows (celui du PATH système, souvent

`C:\Users\utilisateur\AppData\Local\Programs\Python\Python312\python.exe`) au lieu de celui de votre environnement virtuel.

Conséquence : la tâche s'exécute "sans erreur" visible dans le Planificateur (statut "Réussite"), mais le script plante en réalité dès le premier `import pandas`, car ce `python.exe` —là ne connaît pas les paquets installés dans votre venv. Comme le Planificateur ne montre pas la sortie erreur par défaut, ce genre de bug passe facilement inaperçu pendant plusieurs jours.

Réflexe à avoir : pointez toujours explicitement vers le `python.exe` situé dans `...\votre_projet\venv\Scripts\python.exe`, jamais vers l'exécutable global — et pensez à rediriger les logs de votre script vers un fichier (`logging` ou un simple `print` redirigé) pour pouvoir diagnostiquer une exécution silencieusement ratée.

6.2 Mac/Linux — cron

`cron` est le planificateur de tâches historique des systèmes Unix (Linux, macOS). Il exécute des commandes à intervalles réguliers, définis dans un fichier appelé **crontab** (« cron table »).

LES 5 CHAMPS D'UNE LIGNE CRON

Chaque ligne de crontab suit le format suivant :

```
* * * * * commande à exécuter
- - - - -
| | | | |
| | | | +---- jour de la semaine (0-6, 0=dimanche)
| | | +----- mois (1-12)
| | +----- jour du mois (1-31)
| +----- heure (0-23)
+----- minute (0-59)
```

CHAMP	VALEURS POSSIBLES	EXEMPLE	SIGNIFICATION
Minute	0-59	0	à la minute pile
Heure	0-23	8	8h du matin
Jour du mois	1-31	*	tous les jours du mois
Mois	1-12	*	tous les mois
Jour de la semaine	0-6 (0=dimanche)	*	tous les jours de la semaine

L'astérisque ***** signifie « à chaque valeur possible » (pas de restriction). On peut aussi utiliser des listes (**1,15**), des plages (**1-5**) ou des pas (***/2**).

EXEMPLE CONCRET : EXÉCUTER UN SCRIPT TOUS LES JOURS À 8H

```
0 8 * * * /usr/bin/python3 /home/utilisateur/scripts/rapport_quotidien.py
```

Lecture : minute **0**, heure **8**, tous les jours du mois, tous les mois, tous les jours de la semaine → **tous les jours à 8h00 pile.**

ÉDITER ET CONSULTER SA CRONTAB

Pour ouvrir l'éditeur de crontab de l'utilisateur courant :

```
BASH
crontab -e
```

La première fois, le système peut demander de choisir un éditeur (nano est le plus simple pour débuter). On ajoute la ligne de tâche à la fin du fichier, on sauvegarde, et cron recharge automatiquement la configuration.

Pour vérifier ce qui est actuellement planifié :

```
BASH
crontab -l
```

Pour supprimer toutes les tâches planifiées (à utiliser avec précaution) :

```
BASH
crontab -r
```

PIÈGE N°1 : CRON TOURNE DANS UN ENVIRONNEMENT MINIMAL

Contrairement à un terminal interactif, cron ne charge ni le `.bashrc`, ni le `.zshrc`, ni les alias, ni un `PATH` complet. Une commande comme `python3 script.py` qui fonctionne très bien dans un terminal peut échouer silencieusement dans cron, car :

- `python3` n'est pas trouvé (le `PATH` de cron est très réduit, souvent juste `/usr/bin:/bin`) ;
- un chemin relatif (`./script.py` ou `data/fichier.csv`) ne pointe pas vers l'endroit attendu, car le répertoire de travail par défaut de cron n'est pas celui du script.

La règle à retenir : toujours utiliser des chemins absolus, aussi bien pour l'interpréteur Python que pour le script et les fichiers qu'il manipule.

Pour trouver le chemin absolu de son interpréteur Python :

```
BASH
which python3
# /usr/bin/python3 (ou /home/utilisateur/.pyenv/shims/python3, selon
l'installation)
```

Et l'utiliser tel quel dans la ligne de crontab. Si le script lui-même utilise des chemins relatifs en interne, on peut aussi forcer le répertoire de travail avec `cd` avant la commande :

```
0 8 * * * cd /home/utilisateur/scripts && /usr/bin/python3
rapport_quotidien.py
```

PIÈGE N°2 : LA SORTIE DE CRON N'EST AFFICHÉE NULLE PART

Par défaut, cron n'affiche rien à l'écran (il n'y a pas d'écran !). Si le script plante ou affiche des messages avec `print()`, cette sortie part... dans le vide (ou dans un mail local que personne ne consulte). Résultat typique : un script qui échoue tous les jours depuis trois semaines sans que personne ne s'en aperçoive.

La règle à retenir : toujours rediriger la sortie standard et les erreurs vers un fichier de log, avec `>>` (ajout à la fin du fichier) et `2>&1` (redirige aussi les erreurs, pas seulement les messages normaux) :

```
0 8 * * * /usr/bin/python3 /home/utilisateur/scripts/rapport_quotidien.py >>
/home/utilisateur/scripts/log_rapport.txt 2>&1
```

Ainsi, en cas de souci, il suffit de consulter le fichier de log (`tail -n 50 /home/utilisateur/scripts/log_rapport.txt`) pour voir immédiatement si la dernière exécution a réussi ou pourquoi elle a échoué. Cette ligne complète, qui réunit chemins absolus et journalisation, est le format à retenir en pratique.

6.3 Directement en Python — la librairie `schedule`

Cron (ou le Planificateur de tâches Windows) délègue la planification au système d'exploitation : c'est lui qui réveille le script au bon moment. Il existe une alternative plus légère, utile pour des scripts autonomes, des prototypes ou des environnements où l'on ne veut pas toucher à la crontab : la librairie `schedule`, qui gère la planification *à l'intérieur même* du script Python, tant que celui-ci tourne.

Installation :

```
BASH
pip install schedule
```

EXEMPLE COMPLET ET FONCTIONNEL

PYTHON

```
import schedule
import time
from datetime import datetime

def job():
    """La tâche à exécuter périodiquement."""
    print(f"[{datetime.now()}] Exécution du job planifié...")
    # Ici : extraction de données, appel API, mise à jour d'un fichier, etc.

# Planifie le job tous les jours à 08h00
schedule.every().day.at("08:00").do(job)

# Boucle principale : le script doit rester en cours d'exécution
while True:
    schedule.run_pending()
    time.sleep(60)
```

Le principe est simple :

- `schedule.every().day.at("08:00").do(job)` enregistre `job` dans une liste interne de tâches planifiées, avec sa règle de récurrence.
- La boucle `while True` est indispensable : à chaque itération, `schedule.run_pending()` vérifie si une tâche planifiée est arrivée à échéance et, si oui, l'exécute.
- `time.sleep(60)` évite de solliciter le processeur en boucle infinie ; on peut vérifier une fois par minute que tout va bien, ce qui est largement suffisant pour la plupart des usages.

LA FLEXIBILITÉ DE L'API FLUIDE

L'un des atouts de `schedule` est la lisibilité de son API, construite comme une phrase en anglais. Quelques variantes possibles :

PYTHON

```
# Toutes les 10 minutes
schedule.every(10).minutes.do(job)

# Tous les lundis à 09h00
schedule.every().monday.at("09:00").do(job)
```

On peut combiner autant de règles que nécessaire : chaque appel à

`schedule.every(...).do(...)` ajoute une nouvelle tâche indépendante à la file, toutes vérifiées à chaque passage dans `run_pending()`.

LE VRAI PIÈGE À CONNAÎTRE

Contrairement à cron, au Planificateur de tâches Windows ou à GitHub Actions, `schedule` ne fonctionne que tant que le processus Python reste actif. Concrètement :

- Si vous fermez le terminal, arrêtez le script, ou éteignez la machine, plus rien ne se déclenche — il n'y a aucun mécanisme externe qui prend le relais.
- Il faut donc soit laisser le script tourner en permanence (par exemple dans un terminal dédié, un service, ou un conteneur), soit accepter que la planification s'arrête dès que le processus s'arrête.

C'est la différence fondamentale avec cron, le Planificateur de tâches Windows ou GitHub Actions : ces derniers reposent sur un ordonnanceur externe (le système d'exploitation ou une plateforme cloud) qui démarre le script au bon moment, sans qu'un processus Python doive rester allumé en continu entre deux exécutions. `schedule` est donc plutôt adapté à des scripts de longue durée déjà en cours d'exécution (un service, un bot, une application), et moins à des tâches ponctuelles qu'on voudrait déclencher "à distance" sans rien laisser tourner.

Et en R ? R n'a pas d'équivalent direct de `schedule` en usage courant. La pratique R habituelle est de s'appuyer directement sur `cron` (Mac/Linux) ou le Planificateur de tâches (Windows) décrits ci-dessus — il suffit de remplacer `python3 script.py` par `Rscript script.R` dans la commande planifiée. Des packages comme `cronR` ou `taskscheduleR` existent, mais ce sont de simples interfaces qui créent une entrée cron/Task Scheduler pour toi depuis R — le mécanisme sous-jacent reste le même.

6.4 Dans le cloud — GitHub Actions

Jusqu'ici, tous les automatismes qu'on a vus (Planificateur de tâches Windows, `cron` sur macOS/Linux) ont un point commun : **ils tournent sur ton ordinateur**. Si ta machine est éteinte, en veille, ou juste pas connectée à internet à l'heure prévue, le script ne s'exécute pas. Pour un script qui doit tourner tous les jours à 8h, ça veut dire laisser ton ordinateur allumé en permanence — pas très pratique.

La solution : faire tourner le script **dans le cloud**, sur des serveurs qui appartiennent à GitHub, et qui ne dorment jamais. C'est exactement ce que permet **GitHub Actions**.

LE PRINCIPE

GitHub Actions est un service intégré à GitHub qui permet d'exécuter du code automatiquement en réaction à des événements : un `push`, une `pull request`, ou — ce qui nous intéresse ici — **un horaire programmé** (`schedule`).

Concrètement :

1. Tu écris un fichier de configuration au format YAML dans ton dépôt GitHub, à l'emplacement `.github/workflows/`.
2. Ce fichier décrit *quand* le job doit se lancer et *quoi* faire (installer Python, installer les dépendances, exécuter ton script).
3. GitHub se charge du reste : il démarre une machine virtuelle temporaire à l'heure dite, exécute les commandes, puis détruit la machine.

Tu n'as besoin de rien d'autre que ton dépôt GitHub — pas de serveur à louer, pas de carte bancaire (dans la limite du quota gratuit, largement suffisant pour un script quotidien).

LE FICHIER DE WORKFLOW

Reprenons le script d'automatisation de ce module (celui qui interroge une API et enregistre les résultats). Pour le faire tourner chaque jour via GitHub Actions, on crée le fichier suivant à la racine du dépôt :

YAML

```
# .github/workflows/automatisation.yml

name: Automatisation quotidienne

on:
  schedule:
    - cron: '0 8 * * *' # tous les jours à 8h UTC
  workflow_dispatch: # permet de lancer le workflow manuellement

jobs:
  run-script:
    runs-on: ubuntu-latest

    steps:
      - name: Récupérer le code du dépôt
        uses: actions/checkout@v4

      - name: Installer Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Installer les dépendances
        run: pip install -r requirements.txt

      - name: Exécuter le script
        run: python mon_script.py
```

Décortiquons les éléments clés :

- `on: schedule: - cron: '0 8 * * *'` : déclenche le workflow selon une expression cron classique (voir section 6.2 ci-dessus). Ici, `0 8 * * *` signifie « à la minute 0, de l'heure 8, tous les jours ».
- `workflow_dispatch:` : cette ligne, souvent oubliée, ajoute un bouton « **Run workflow** » dans l'onglet *Actions* de GitHub. Elle permet de déclencher le workflow **manuellement**, à tout moment, sans attendre l'horaire programmé. C'est extrêmement pratique pour tester que tout fonctionne avant de laisser tourner le planning automatique — ne t'en prive pas.
- `actions/checkout@v4` : cette étape clone ton dépôt sur la machine virtuelle temporaire. Sans elle, la machine serait vide et n'aurait pas accès à ton script.
- `actions/setup-python@v5` : installe la version de Python demandée sur la machine virtuelle.

- `pip install -r requirements.txt` : installe les librairies dont ton script a besoin (`requests`, `pandas`, etc.). C'est pour ça qu'il est indispensable d'avoir un fichier `requirements.txt` à jour dans ton dépôt (voir Module 13).
- `python mon_script.py` : lance enfin ton script, exactement comme tu le ferais en local.

À retenir : une fois ce fichier poussé (`git push`) dans `.github/workflows/`, rends-toi dans l'onglet **Actions** de ton dépôt GitHub. Tu y verras apparaître ton workflow, avec la possibilité de consulter les logs de chaque exécution — très utile pour déboguer si quelque chose se passe mal.

⚠ PIÈGE CLASSIQUE : LE FUSEAU HORAIRE UTC

C'est la confusion la plus fréquente chez les débutants, alors soyons très clairs :

Les horaires `cron` de GitHub Actions sont toujours exprimés en heure UTC (temps universel coordonné), jamais dans ton fuseau horaire local.

Autrement dit, `cron: '0 8 * * *'` ne veut **pas** dire « tous les jours à 8h chez moi ». Ça veut dire « tous les jours à 8h à Londres en hiver (UTC = heure de Londres hors heure d'été) ». Il faut donc convertir toi-même :

VILLE	DÉCALAGE PAR RAPPORT À UTC	POUR UN SCRIPT À 8H <i>HEURE LOCALE</i> , IL FAUT ÉCRIRE
Abidjan (Côte d'Ivoire)	UTC+0 (toute l'année, pas de changement d'heure)	<code>cron: '0 8 * * *'</code>
Paris, en hiver (heure d'hiver, UTC+1)	UTC+1	<code>cron: '0 7 * * *'</code>
Paris, en été (heure d'été, UTC+2)	UTC+2	<code>cron: '0 6 * * *'</code>

Deux points de vigilance supplémentaires :

- **Abidjan a un décalage constant** (UTC+0 toute l'année, pas de changement d'heure été/hiver), donc une seule valeur de cron suffit à l'année longue.
- **Paris (et la France en général) change d'heure deux fois par an** (heure d'été / heure d'hiver). Si tu es en France et que tu veux une heure locale précise à la minute près toute l'année, tu devras théoriquement ajuster ton cron deux fois par an — ou accepter un décalage d'une heure pendant une partie de l'année.
- GitHub précise aussi, dans sa documentation, que les workflows programmés peuvent démarrer avec quelques minutes de retard en cas de forte charge sur ses serveurs. Ne compte donc pas sur une exécution à la seconde près.

Réflexe à avoir : chaque fois que tu écris un `cron` dans un workflow GitHub Actions, demande-toi explicitement « quelle heure UTC correspond à l'heure locale que je veux ? », et ajoute un commentaire dans le YAML pour t'en souvenir (comme dans l'exemple plus haut : `# tous les jours à 8h UTC`).

STOCKER UNE CLÉ D'API EN TOUTE SÉCURITÉ AVEC LES SECRETS

Si ton script a besoin d'une clé d'API, il est **hors de question** de l'écrire en clair dans le script ou dans le fichier YAML. Ce fichier est versionné avec Git et visible publiquement si ton dépôt est public — quiconque pourrait alors utiliser ta clé à ta place (et parfois à tes frais).

GitHub propose pour cela un mécanisme de **secrets** : des variables chiffrées, stockées côté GitHub, invisibles dans le code et jamais affichées dans les logs.

Étape 1 — Créer le secret sur GitHub

1. Va sur la page de ton dépôt GitHub.
2. Clique sur **Settings** (Paramètres).
3. Dans le menu de gauche, va dans **Secrets and variables** → **Actions**.
4. Clique sur **New repository secret**.
5. Donne un nom à ta variable, par exemple `API_KEY`, colle la valeur de ta clé, puis valide.

Ta clé est maintenant stockée de façon chiffrée par GitHub. Elle n'apparaîtra jamais dans le code du dépôt, ni dans l'historique Git, ni dans les logs d'exécution des workflows.

Étape 2 — Référencer le secret dans le workflow

Dans le fichier `.github/workflows/automatisation.yml`, on transmet le secret au script sous forme de **variable d'environnement**, grâce au bloc `env:` :

YAML

```
- name: Exécuter le script
  env:
    API_KEY: ${ secrets.API_KEY }
  run: python mon_script.py
```

Côté Python, le script va simplement lire cette variable d'environnement :

PYTHON

```
import os

cle_api = os.environ["API_KEY"]
```

Le point essentiel à retenir : **la clé n'est jamais écrite en dur nulle part dans le dépôt**. Elle vit uniquement dans les paramètres sécurisés de GitHub, et n'est injectée dans l'environnement d'exécution qu'au moment où le workflow tourne, via `secrets.API_KEY`.

Bonne pratique : utilise ce système de secrets pour **toute** information sensible — clés d'API, mots de passe, tokens d'accès, identifiants de base de données. La règle est la même que pour un fichier `.env` en local : si c'est secret, ça ne doit jamais se retrouver dans un fichier suivi par Git.

RÉSUMÉ EXPRESS

ÉLÉMENT DU WORKFLOW	RÔLE
<code>on: schedule: cron:</code>	Définit l'horaire d'exécution automatique, en UTC
<code>workflow_dispatch:</code>	Ajoute un déclenchement manuel, pratique pour tester
<code>actions/checkout@v4</code>	Récupère le code du dépôt sur la machine virtuelle
<code>actions/setup-python@v5</code>	Installe Python
<code>pip install -r requirements.txt</code>	Installe les dépendances du script
<code>secrets.API_KEY</code>	Injecte une clé d'API de façon sécurisée, sans l'exposer dans le code

💡 **En résumé** : pour un script qui doit tourner tous les jours sans que ton ordinateur reste allumé, un job planifié dans le cloud (GitHub Actions, par exemple — tu as déjà un dépôt GitHub depuis le Module 13) est l'option la plus robuste. C'est la version « professionnelle » de l'automatisation, celle qu'on retrouve dans la plupart des pipelines de data en entreprise.

7. Envelopper le script dans une mini app Streamlit

Streamlit transforme un script Python en application web interactive, partageable, en quelques lignes — sans rien connaître au développement web.

Installer

BASH

```
pip install streamlit
```

Une première app

Crée un fichier `app.py` :

PYTHON

```
import streamlit as st
import pandas as pd

st.title("📈 Taux de change FCFA")

devise_source = st.selectbox("Devise source", ["USD", "EUR", "GBP"])

if st.button("Récupérer le taux actuel"):
    taux = recuperer_taux(devise_source, ("XOF",))
    st.metric(label=f"{devise_source} → XOF", value=f"{taux['XOF']:.2f} FCFA")
```

Lancer l'app

BASH

```
streamlit run app.py
```

Ton navigateur s'ouvre automatiquement sur une petite application web : un menu déroulant pour choisir la devise, un bouton qui déclenche l'appel API en direct, et le résultat affiché immédiatement. C'est le même script d'automatisation que tu as écrit en section 2-3 — Streamlit ne fait qu'ajouter une couche de présentation par-dessus.

 **Pont vers le Mini-Projet C** : c'est exactement le schéma que tu vas construire pour le pipeline complet — API (ce module) → nettoyage (Modules 14-15) → mini-app Streamlit (ce module).

✓ Résumé du module

CONCEPT	CE QU'IL FAUT RETENIR
API	Point d'accès qui répond à des requêtes HTTP et renvoie des données, généralement en JSON
Types d'API	REST (le plus courant), SOAP (systèmes historiques/bancaires), GraphQL (requête sur mesure), WebSocket (temps réel/streaming)
<code>requests.get()</code> / <code>GET()</code> (httr)	Fait une requête GET vers une API
<code>.json()</code> / <code>fromJSON(content(...))</code>	Convertit la réponse en dictionnaire/liste exploitable
Statut HTTP 200 ≠ succès garanti	Certaines APIs renvoient une erreur dans le corps de la réponse JSON, avec un statut 200 — toujours vérifier le contenu, pas seulement le statut
<code>raise_for_status()</code> / <code>stop_for_status()</code>	Lève une exception si le statut HTTP est une erreur (4xx/5xx) — ne couvre pas les erreurs "silencieuses" ci-dessus
Journaliser (logger)	Ajouter une ligne horodatée à un fichier à chaque exécution, pour construire un historique
Planifier	Task Scheduler (Windows), <code>cron</code> (Mac/Linux), librairie <code>schedule</code> , ou un job cloud (GitHub Actions)
Streamlit	Transforme un script Python en app web interactive et partageable, en quelques lignes

🧠 Quiz — Vérifie ta compréhension

Q1. Une API te renvoie un statut HTTP 200, mais le corps de la réponse contient `{"result": "error", ...}`. Que dois-tu en conclure ?

- a) La requête a réussi, tu peux utiliser les données sans vérification
- b) `raise_for_status()` aurait dû lever une exception automatiquement
- c) Il faut aussi vérifier le contenu de la réponse — certaines APIs signalent une erreur dans le corps, pas seulement via le statut HTTP

👉 Voir la réponse

✓ c) — `raise_for_status()` (b) ne réagit qu'aux codes de statut HTTP d'erreur (4xx/5xx) ; un statut 200 avec une erreur "cachée" dans le JSON ne déclenchera jamais cette exception. Il faut toujours vérifier explicitement le contenu de la réponse.

Q2. Pourquoi journaliser (logger) chaque exécution d'un script d'automatisation dans un fichier, plutôt que de juste afficher le résultat à l'écran ?

- a) Ça rend le script plus rapide
- b) Ça construit un historique exploitable plus tard, même si personne n'a regardé l'écran au moment de l'exécution
- c) C'est obligatoire pour que Python fonctionne

👉 Voir la réponse

✓ b) — Un script planifié (cron, Task Scheduler) tourne souvent sans personne devant l'écran. Journaliser dans un fichier garantit que le résultat de chaque exécution reste disponible pour une analyse ultérieure.

Q3. Qu'est-ce que Streamlit ajoute par rapport à ton script Python d'origine ?

- a) Il rend les appels API plus rapides
- b) Une couche de présentation web interactive, sans changer la logique du script
- c) Il remplace la nécessité d'écrire du code Python

 [Voir la réponse](#)

✓ **b)** — Streamlit enveloppe ton code existant (la fonction `recuperer_taux`, par exemple) dans une interface web cliquable — la logique métier ne change pas, seule la présentation devient interactive et partageable.

Q4. Un·e collègue te dit que le CRM de l'entreprise expose son suivi des ventes en temps réel via une connexion permanente qui pousse chaque nouvelle vente dès qu'elle est enregistrée, sans que tu aies besoin de renvoyer une requête. De quel type d'API s'agit-il probablement ?

- a) REST
- b) SOAP
- c) WebSocket

 [Voir la réponse](#)

✓ **c)** — Le WebSocket maintient une connexion permanente où le serveur pousse les mises à jour au fur et à mesure, sans requête/réponse répétée — exactement le scénario "temps réel" décrit. REST (a) fonctionne par requête/réponse ponctuelle (tu dois redemander pour avoir du nouveau), et SOAP (b) est un protocole plus ancien basé sur XML, sans lien avec le temps réel.

Module suivant

Tu sais maintenant automatiser une extraction de données et l'exposer via une mini app. Dans le **Module 20**, on passe au storytelling — comment présenter tes résultats pour convaincre un·e décideur·se.

→ **Module 20 — Storytelling + IA**

